

メディア情報学プログラミング演習
最終レポート
人生100年サバイバル

学籍番号：2411610 氏名：志熊 陽斗 (Model 担当)
学籍番号：xxxxxxx 氏名：北村 ○○ (Controller 担当)
学籍番号：xxxxxxx 氏名：佐藤 ○○ (View 担当)

2026年2月16日

1 概要説明

1.1 実現したプログラムの目的と機能

本グループでは、0 歳から 100 歳までの人生をシミュレーションするノベルゲームを作成した。プレイヤーは主人公となり、幼少期、学生時代、社会人、老後といった人生の各ステージで提示される選択肢を選びながら、100 歳まで生きる人生を目指す。

主な機能は以下の通りである。

- **年齢進行システム**: 0 歳からスタートし、選択肢に応じて時間が経過する。
- **ステータス管理**: 「体力」「ストレス」「所持金」のパラメータを管理し、これらが限界を超えるとゲームオーバー（過労死・破産など）となる。
- **可変時間システム**: 選択肢によって経過する年数が異なる（例：大学進学＝4 年経過、就職＝1 年経過）。これによりリアルな人生設計が可能となっている。
- **ライフイベント分岐**: 年齢や乱数に応じて、多彩なイベント（結婚、昇進、病気など）が発生する。

1.2 作業の分担

本プロジェクトは MVC（Model-View-Controller）アーキテクチャを採用し、以下のように役割分担を行って開発を進めた。

表 1 メンバーの役割分担

担当	氏名	主な作業内容
Model	志熊 陽斗	データ構造設計、ゲームロジック実装、イベントデータ作成
View	佐藤 ○○	GUI 画面（Swing）の作成、画面描画処理
Controller	北村 ○○	メインループ制御、イベントリスナー実装、Model と View の連携

2 設計方針

2.1 基本構想とアルゴリズム

本プログラムは、ユーザの選択がステータスと未来のイベントに影響を与える「状態遷移型」のシミュレーションゲームとして設計した。

最も特徴的なアルゴリズムは「**可変時間ステップ**」である。選択肢を選んだとき、すべての選択肢で経過時間を等しくするのではなく、選択した行動の重みに応じて経過時間を動的に変化させるデータ構造を採用した。

2.2 クラス構成 (MVC モデル)

Java のオブジェクト指向特性を活かし、機能ごとにクラスを分割する MVC モデルを採用した。以下に主要なクラス構成図を示す。

[クラス図概略]

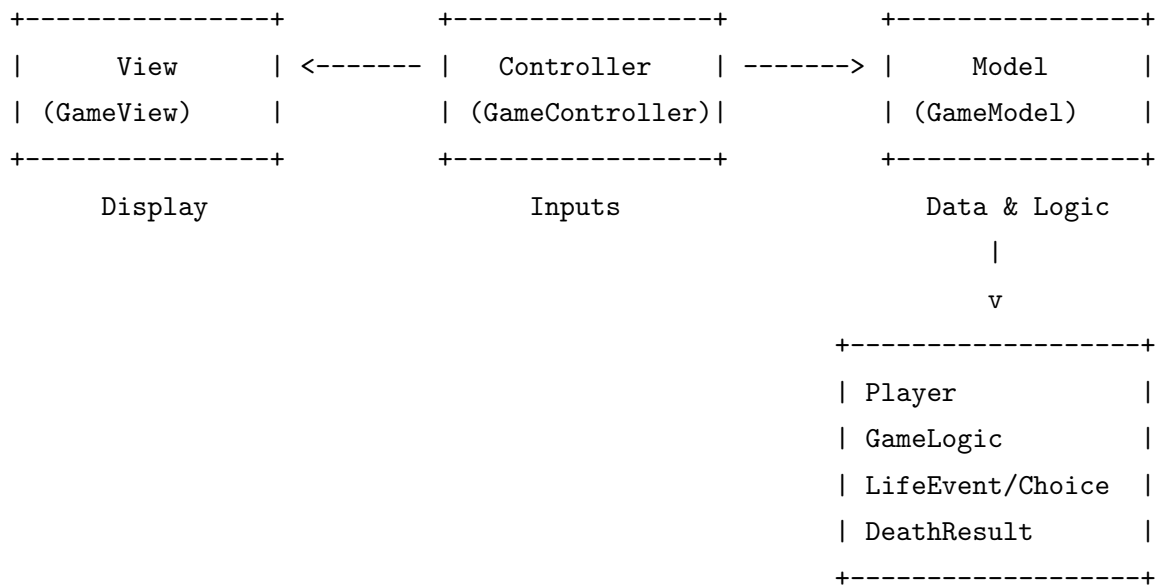


図 1 本システムのクラス構成図 (MVC 連携)

なお、クラス図において Player, GameLogic などは本演習で新規に作成したクラスである。GUI コンポーネント (JFrame, JButton 等) は Java 標準ライブラリを利用している。

3 プログラムの説明

本章では、実装した各クラスの詳細について説明する。開発は MVC モデルに基づき分担して行ったため、以下に各担当者が作成したパートごとに記述する。

3.1 Model 層（データとロジック）

文責：志熊 陽斗 (2411610)

Model 層は、ゲームの状態管理、イベントデータの定義、およびゲーム進行のルールを担当する部分である。View（表示）や Controller（入力）から独立させることで、仕様変更に強い設計を目指した。以下に、私が設計・実装した主要なクラスについて、コードの挙動と共に詳細に解説する。

3.1.1 Player クラス

プレイヤーの現在のステータス（年齢、体力、ストレス、所持金、生死状態）を保持するクラスである。フィールドはすべて `private` で隠蔽し、外部からの不正なアクセスを防いでいる。

工夫した点：

- **データの整合性保証:** 単に値を足し引きするだけでなく、上限（100）や下限（0）のチェックをメソッド内で行うことで、体力が 100 を超えてしまうなどの異常なステータスが発生しないように設計した。
- **可変時間への対応:** 年齢の加算を固定値ではなく引数で受け取る仕様にし、イベントごとの時間経過の違いに対応させた。

```
1 public class Player {
2     // 状態フィールド（外部から直接変更不可）
3     private int age;
4     private int health;
5     private boolean isAlive;
6
7     // 年齢を加算するメソッド
8     public void incrementAge(int years) {
9         this.age += years; // 引数で渡された年数分だけ加算する
10    }
11
12    // 体力を増減させるメソッド
13    public void changeHealth(int amount) {
14        this.health += amount; // 値を加算（マイナスの
15                               // 場合は減少）
16        if (this.health > 100) { // 上限チェック
17            this.health = 100; // 100を超えたら100に補
```

```

17         }
18         // 下限チェックはここでは行わず、
           checkVitalityに委ねる設計とした
19     }
20
21     // 生死判定を行うメソッド
22     public void checkVitality() {
23         // 体力が0以下、またはストレスが100以上になったか判定
24         if (this.health <= 0 || this.stress >= 100) {
25             this.isAlive = false;           // 生存フラグを
           false (死亡)に変更
26         }
27     }
28 }

```

ソースコード 1 Player.java (ステータス操作の抜粋)

コードの詳細解説：

- **9-11 行目:** incrementAge メソッドでは、単純な age++ ではなく、引数 years を受け取って加算している。これにより、可変時間の処理を Model 層の内部で完結させている。
- **14-19 行目:** changeHealth メソッドでは、15-17 行目で「上限キャップ処理」を行っている。これにより、回復イベントが連続しても体力が異常値になるバグを防いでいる。
- **22-27 行目:** checkVitality メソッドは、ゲームオーバー条件を集約した場所である。複数のステータス（体力とストレス）を複合的に監視し、どちらか一方でも限界を超えれば即座に isAlive を false にするロジックとなっている。

3.1.2 Choice クラス / LifeEvent クラス

イベントと、それに紐づく選択肢を表現するクラス群である。本ゲームの最大の特徴である選択による時間経過の変化をデータ構造レベルでサポートしている。こうすることで各イベントを木構造のようにつなげて実装するよりも構造を簡単にできる。

工夫した点：

- **経過年数のデータ化:** Choice クラスに ageDelta フィールドを持たせることで、選択肢と時間の重みをセットで管理できるようにした。これにより、プログラム側でこのイベントの時は何年進むとハードコーディングする必要がなくなり、データの独立性が高まった。

```

1 class Choice {
2     public String text;           // 選択肢の文言
3     public int healthDelta;       // 体力変動値
4     public int ageDelta;          // 経過年数 (重要)

```

```

5
6 // コンストラクタ
7 public Choice(String text, int h, ..., int ageDelta) {
8     this.text = text;
9     this.healthDelta = h;
10    this.ageDelta = ageDelta; // 引数で受け取った年数を保持
11 }
12
13 // UI表示用にテキストを整形する
14 public String getText() {
15     // ボタンに「(+4年)」のような情報を付加して返す
16     return text + "␣(+ " + ageDelta + "年)";
17 }
18 }

```

ソースコード 2 Choice.java (データ構造)

コードの詳細解説：

- 4 行目: `int ageDelta` フィールドが、このクラスの核である。ここには「大学に行く」なら 4、「休学する」なら 1 といった整数値が格納される。
- 15-17 行目: `getText` メソッドは View 層のための補助メソッドである。単に「大学に行く」と表示するだけでなく、「大学に行く (+4 年)」と動的に文字列を結合して返すことで、プレイヤーに対して選択のリスク（時間の経過）を可視化している。

3.1.3 GameLogic クラス（ゲームロジック）

現在の年齢を受け取り、適切なイベントを生成して返すクラスである。

工夫した点：

- 生成ロジック: 幼少期は年齢固定イベント、成年期以降はランダムイベントと生成方式を切り替えることで、成長過程のリアリティとゲームとしてのリプレイ性を両立させた。
- 階層的な条件分岐: 巨大な `if-else` 文になるのを防ぐため、年齢帯ごとにプライベートメソッド（`getChildhoodEvent`, `getAdultEvent` など）に分割し、可読性を維持した。

```

1 class GameLogic {
2     // 年齢に応じたイベントを振り分けるメインメソッド
3     public LifeEvent getEventForAge(int age) {
4         if (age < 23) {
5             return getChildhoodEvent(age); // 22歳以下は固定イベント
6         } else if (age < 65) {
7             return getAdultEvent(age); // 64歳以下は社会人ランダム
8         }
9     }
10 }

```

```

8         }
9         // ... (以下省略) ...
10    }
11
12    // 社会人時代のランダムイベント生成
13    private LifeEvent getAdultEvent(int age) {
14        LifeEvent event;
15        double rnd = Math.random(); // 0.0~1.0の乱数を生成
16
17        // 乱数の値によって発生するイベントを分岐させる
18        if (rnd < 0.4) {
19            event = new LifeEvent("【" + age + "歳】仕事のトラブ
                ル...");
20            // ... (選択肢の追加) ...
21        } else if (rnd < 0.7) {
22            event = new LifeEvent("【" + age + "歳】結婚式の招待
                状...");
23        } else {
24            event = new LifeEvent("【" + age + "歳】昇進のチャンス!");
25        }
26        return event;
27    }
28 }

```

ソースコード 3 GameLogic.java (イベント生成の抜粋)

コードの詳細解説：

- **3-10 行目:** `getEventForAge` メソッドは、年齢という単一の入力に対し、適切なイベントオブジェクトを返す役割を果たしている。ここで大まかな年代による分岐を行っている。
- **15 行目:** `Math.random()` を用いて乱数を生成している。この値はメソッドが呼ばれるたびに变化するため、同じ年齢であっても、プレイするたびに仕事のトラブルが起きたり昇進したりと展開が変わる仕組みを実現している。
- **18-24 行目:** 生成した乱数 `rnd` の範囲 (0.0~0.4, 0.4~0.7, それ以外) によってイベントを決定している。この閾値を調整することで、イベントの発生確率 (レアイベントなど) をコントロールできる設計となっている。

3.1.4 GameModel クラス / DeathResult クラス (統合・ファサード)

Model 層の全機能を統合し、Controller 層への単一の窓口を提供するクラスである。また、メソッドの戻り値を `DeathResult` クラス (生死状態・メッセージ・最終年齢をまとめたデータクラス) とすることで、情報の受け渡しを簡潔にした。

工夫した点：

- 責務の完全な分離: Controller 層は applyChoice メソッドを呼ぶだけでよく、内部で Player がどう計算され、GameLogic がどう判定しているかを知る必要がない設計とした。
- 安全なトランザクション処理: 「ステータス更新」→「死亡判定」→「年齢加算」という順序をメソッド内で強制することで、死んでいるのに年齢が増えるといった矛盾を防いでいる。

```
1 class GameModel {
2     // 選択肢の結果を適用する一連の流れをカプセル化
3     public DeathResult applyChoice(Choice choice) {
4         // 1. ステータス反映
5         player.changeHealth(choice.healthDelta);
6         player.changeStress(choice.stressDelta);
7         player.changeMoney(choice.moneyDelta);
8
9         // 2. 死亡判定（即死チェック）
10        player.checkVitality();
11        if (!player.isAlive()) {
12            // 死亡していた場合、ここで処理を中断して結果を返す
13            return new DeathResult(
14                DeathResult.Type.DEAD, "過労
15                死...", "...", player.getAge()
16            );
17        }
18
19        // 3. 年齢加算（生存時のみ時間を進める）
20        player.incrementAge(choice.ageDelta);
21
22        // 4. クリア判定（100歳到達）
23        if (player.getAge() >= Player.HUMAN_LIFESPAN) {
24            return new DeathResult(DeathResult.Type.DEAD, "大往
25            生", "...", player.getAge());
26        }
27
28        // 5. 生存継続
29        return new DeathResult(DeathResult.Type.ALIVE, "", "",
30            player.getAge());
31    }
32 }
```

ソースコード 4 GameModel.java（ファサードの実装詳細）

コードの詳細解説：

- 5-7 行目: まず最初に、選択された行動によるステータスの増減（体力、ストレス、所持金）を Player オブジェクトに反映させる。
- 10-16 行目: ステータス変動後、即座に checkVitality を呼び出して生死確認を行う。も

し死亡している場合は、**年齢を加算する前に**メソッドを終了させている。これは過労死した時点で時間は止まるというリアリティを持たせるために必要なロジックである。

- **19 行目:** 生存が確認された場合のみ、`incrementAge` を実行して時間を進める。ここで初めて次の年齢が確定する。
- **22-24 行目:** 時間が進んだ結果、寿命に達したかどうかを判定する。達していれば「大往生」として結果を返す。

3.2 View 層 (GUI 画面表示)

文責：佐藤 ○○

(※ここに佐藤さんの担当箇所の説明が入ります。Swing コンポーネントの配置や、イベント内容のテキストエリア表示などの工夫点について記述されます。)

3.3 Controller 層 (制御フロー)

文責：北村 ○○

(※ここに北村さんの担当箇所の説明が入ります。ActionListener の実装や、Model と View の橋渡し処理について記述されます。)

4 実行例

本プログラムの主要な機能について、実際の実行画面を用いて説明する。

4.1 ゲーム開始～幼少期

プログラムを起動すると、0 歳からゲームが開始される。画面上部には現在の年齢、体力 (HP)、ストレス、所持金が表示され、中央にはイベントテキスト、下部には選択肢ボタンが表示される。

[画面イメージ: 0 歳]

年齢: 0 歳

HP: 50 | ストレス: 0 | 所持金: 0 円

【0 歳】誕生。光に包まれて、あなたの人生が始まります。

[大声で泣いて自己主張 (+3 年)]

[静かに親を見つめる (+3 年)]

図 2 ゲーム開始直後の画面 (0 歳)

ボタンには「(+3 年)」と表示されており、選択すると 3 歳へジャンプすることが視覚的に分かるようになっている。

4.2 人生の分岐点 (18 歳～)

18 歳のイベントでは、進路選択が行われる。ここで「大学進学 (+1 年)」を選ぶか、「就職 (+5 年)」を選ぶかによって、その後の 20 代のイベント内容と経過速度が変化する。

[画面イメージ: 18 歳]

年齢: 18 歳

HP: 80 | ストレス: 40 | 所持金: 0 円

【18 歳】進路選択。大人への第一歩。

[大学進学 (奨学金なし) (+1 年)]

[大学進学 (奨学金あり) (+1 年)]

[就職して自立 (+5 年)]

図 3 進路選択の画面 (18 歳)

4.3 ゲームオーバーとエンディング

体力が 0 になるか、ストレスが 100 に達するとゲームオーバーとなる。一方、無事に 100 歳を迎えると「大往生」となり、クリアダイアログが表示される。

5 考察

本プログラムの開発において、当初は0歳から100歳までの人生シミュレーションを、各ライフイベントをノード、選択肢をエッジとした木構造として実装することを検討した。しかし、100年という長期スパンにおいて全ての分岐を木構造で管理しようとする、階層が深くなりすぎ、イベント総数が指数関数的に増加して管理が困難になるという問題に直面した。そこで、本開発では代替案として、Model 層の `Choice` クラスに `ageDelta` (経過年数) を導入し、現在の年齢というステータスを参照してイベントを呼び出す可変時間ステップ方式を採用した。この設計の結果、人生が分岐し、数年後に再び同じキャリアパスに合流するような複雑なシナリオであっても、分岐ごとの加算年数を調整してターゲットとなる年齢に合わせるだけで実装できた。これにより、複雑なポインタ管理をすることなく擬似的な分岐構造を実現でき、木構造を採用するよりも設計を大幅に簡潔化できたと言える。また、単調になりがちな社会人パートにおいて、5年刻みの時間経過と乱数によるイベント選出を組み合わせた点は、実装コストを抑えつつ、結婚、昇進、病気などのイベントによるプレイごとの多様性を確保し、ゲームとしての面白さを高める上で非常に効果的であった。

一方で、今回の設計手法には課題も残った。第一にイベントの挿入・変更コストである。木構造であれば、イベント間に新しいイベントを挿入する場合、親ノードと子ノードのリンクを書き換えるだけで済む。しかし、今回採用した年齢に依存する方式では、中間にイベントを追加しようすると、後続のイベント発生年齢が重複しないよう全て書き換える必要が生じた。拡張性・保守性の観点では、木構造のような相対的な参照方式に分があったと言える。第二にストーリーの連続性である。現在はイベントの発生条件が年齢と乱数のみに依存しているため、イベント間の因果関係が希薄である。これを解決するためには、単に年齢を進めるだけでなく、`Player` クラスに学歴や既婚といった状態フラグを持たせ、`GameLogic` でのイベント選出時にそれらを参照させるなどの対策が考えられる。これにより、今回採用した可変時間ステップの簡潔さを維持しつつ、木構造のような文脈のあるストーリー展開を実現することが今後の課題である。

6 各自の反省と感想

6.1 志熊 陽斗 (Model 担当)

私は Model 層を担当し、主にデータ構造とロジックの実装を行った。最も苦労したのは、View や Controller から「いかにして内部ロジックを隠蔽するか」という点である。当初は Controller から直接 `Player` の値を書き換えるような設計を考えていたが、それだとバグの原因になりやすいと気づき、`GameModel` という窓口クラスを用意して「選択肢を渡せば結果が返ってくる」というシンプルなインターフェースに徹した。結果として、チームメンバーとの連携がスムーズに進み、MVC モデルの利点（分業のしやすさ）を肌で感じる事ができた。オブジェクト指向設計の重要

性を再認識する良い機会となった。

6.2 佐藤 ○○ (View 担当)

(※ここに佐藤さんの感想が入ります。Swing での画面作成の苦労や、Model からデータを受け取って表示する際の連携についての感想などが記述されます。)

6.3 北村 ○○ (Controller 担当)

(※ここに北村さんの感想が入ります。イベントリスナーの実装や、ゲームループの制御についての感想などが記述されます。)

付録 A 付録 1：操作法マニュアル

起動方法

配布された `InfiniteLifeTunnel.jar` をダブルクリックするか、コマンドラインから以下のコマンドを実行してください。

```
java -jar InfiniteLifeTunnel.jar
```

画面の見方

- **上部エリア**: 現在の年齢、体力、ストレス、所持金が表示されます。
- **中央エリア**: 現在発生しているイベントのストーリーが表示されます。
- **下部ボタン**: プレイヤーが取れる行動の選択肢です。「(+3 年)」などの表記は、その選択肢を選んだ場合の経過年数を表します。

遊び方

1. 画面下部の選択肢ボタンをクリックして、人生の選択を行ってください。
2. 選択結果に応じて、ステータス変動し、年齢が進みます。
3. 体力が 0 になるか、ストレスが 100 になるとゲームオーバーです。無理のない選択を心がけましょう。
4. 100 歳まで生き延びることができればゲームクリア（大往生）です。

付録 B 付録 2：プログラムリスト

以下に、本プロジェクトの全ソースコードを掲載する。なお、コンパイルおよび実行には JDK 8 以上が必要である。

1. エントリーポイント (Main.java)

```
1  import javax.swing.SwingUtilities;
2
3  /**
4   * アプリケーションのエントリーポイント
5   */
6  public class Main {
7      public static void main(String[] args) {
8          // SwingアプリはEDT(Event Dispatch Thread)で起動する
9          SwingUtilities.invokeLater(() -> {
10              new GameController();
11          });
12      }
13  }
```

ソースコード 5 Main.java

2. Model 層 (データとロジック)

```
1  /**
2   * プレイヤーの状態（年齢、体力、ストレス、金）を管理するクラス
3   */
4  public class Player {
5      public static final int HUMAN_LIFESPAN = 100; // 寿命
6
7      private int age;
8      private int health;
9      private int stress;
10     private long money;
11     private boolean isAlive;
12
13     public Player() {
14         this.age = 0;           // 0歳スタート
15         this.health = 50;       // 乳幼児期は体力低め
16         this.stress = 0;
17         this.money = 0;
```

```

18         this.isAlive = true;
19     }
20
21     public void incrementAge(int years) {
22         this.age += years;
23     }
24
25     public void changeHealth(int amount) {
26         this.health += amount;
27         if (this.health > 100) this.health = 100;
28     }
29
30     public void changeStress(int amount) {
31         this.stress += amount;
32         if (this.stress < 0) this.stress = 0;
33     }
34
35     public void changeMoney(long amount) {
36         this.money += amount;
37     }
38
39     public void checkVitality() {
40         if (this.health <= 0 || this.stress >= 100) {
41             this.isAlive = false;
42         }
43     }
44
45     // Getter methods
46     public int getAge() { return age; }
47     public boolean isAlive() { return isAlive; }
48     public int getHealth() { return health; }
49     public int getStress() { return stress; }
50     public long getMoney() { return money; }
51 }

```

ソースコード 6 Player.java

```

1 import java.util.ArrayList;
2 import java.util.List;
3
4 /**
5  * 1つの選択肢データを表すクラス
6  */
7 class Choice {
8     public String text;
9     public int healthDelta;

```

```

10     public int stressDelta;
11     public long moneyDelta;
12     public int ageDelta; // 経過年数
13
14     public Choice(String text, int h, int s, long m, int ageDelta
15         ) {
16         this.text = text;
17         this.healthDelta = h;
18         this.stressDelta = s;
19         this.moneyDelta = m;
20         this.ageDelta = ageDelta;
21     }
22
23     public String getText() {
24         return text + "␣(" + ageDelta + "年)";
25     }
26
27     /**
28      * 1つのイベント（場面）を表すクラス
29      */
30     class LifeEvent {
31         private String title;
32         private List<Choice> choices;
33
34         public LifeEvent(String title) {
35             this.title = title;
36             this.choices = new ArrayList<>();
37         }
38
39         public void addChoice(String text, int h, int s, long m, int
40             ageDelta) {
41             choices.add(new Choice(text, h, s, m, ageDelta));
42         }
43
44         public String getTitle() { return title; }
45         public String getText() { return title; }
46         public List<Choice> getChoices() { return choices; }
47     }

```

ソースコード 7 Choice.java / LifeEvent.java

```

1     /**
2      * ゲームの進行結果（生存、死亡、年齢など）を
3      * Controllerへ渡すためのクラス

```

```

4  class DeathResult {
5      public enum Type { ALIVE, DEAD }
6
7      public Type type;
8      public String title;
9      public String message;
10     public int age;
11
12     public DeathResult(Type type, String title, String message,
13         int age) {
14         this.type = type;
15         this.title = title;
16         this.message = message;
17         this.age = age;
18     }
19 }

```

ソースコード 8 DeathResult.java

```

1  /**
2   * 年齢に応じたイベントを生成・選択するクラス
3   */
4  class GameLogic {
5
6      public LifeEvent getEventForAge(int age) {
7          if (age < 23) {
8              return getChildhoodEvent(age);
9          } else if (age < 65) {
10             return getAdultEvent(age);
11          } else if (age < 100) {
12              return getSeniorEvent(age);
13          } else {
14              return getEndingEvent();
15          }
16      }
17
18      // 0歳～22歳：子供・学生時代
19      private LifeEvent getChildhoodEvent(int age) {
20          LifeEvent event = new LifeEvent("イベントエラー");
21
22          switch (age) {
23              case 0:
24                  event = new LifeEvent("
25                      【0歳】誕生。光に包まれて人生が始まります。");
26                  event.addChoice("大声で泣いて自己主張", 5, 0, 0, 3);
27                  event.addChoice("静かに親を見つめる", 0, 0, 0, 3);

```



```

27         break;
28     case 3:
29         event = new LifeEvent("
30             【3歳】自我の芽生え。「イヤイヤ期」です。");
31         event.addChoice("何でも「イヤ!」と拒否", 5, 10, 0, 3);
32         event.addChoice("いい子にして褒められ
33             る", 0, -5, 0, 3);
34         event.addChoice("積み木に熱中する", 0, 0, 0, 3);
35         break;
36     case 6:
37         event = new LifeEvent("【6歳】小学校入学。ランド
38             セルが歩いているようです。");
39         event.addChoice("すぐに隣の子と話す", 5, 5, 0, 2);
40         event.addChoice("緊張して固まる", -2, 10, 0, 2);
41         break;
42     case 8:
43         event = new LifeEvent("
44             【8歳】低学年。習い事を始めますか?");
45         event.addChoice("スイミングに通
46             う", 10, 5, -50000, 2);
47         event.addChoice("ピアノを習う", 0, 5, -100000, 2);
48         event.addChoice("公園で毎日遊ぶ", 5, -5, 0, 2);
49         break;
50     case 10:
51         event = new LifeEvent("【10歳】1/2成人式。");
52         event.addChoice("目立ちたがり屋のリー
53             ダー", 0, 10, 0, 2);
54         event.addChoice("聞き上手なサポーター", 0, 0, 0, 2);
55         event.addChoice("我が道を行く一匹狼", 5, -5, 0, 2);
56         break;
57     case 12:
58         event = new LifeEvent("
59             【12歳】卒業間近。バレンタインの思い出。");
60         event.addChoice("好きな人に告白す
61             る", 5, 20, -5000, 1);
62         event.addChoice("友達とチョコ交換", 0, 0, -3000, 1);
63         event.addChoice("興味ないフリをする", 0, -5, 0, 1);
64         break;
65     case 13:
66         event = new LifeEvent("
67             【13歳】中学生。部活動の厳しさ。");
68         event.addChoice("レギュラー目指して朝
69             練", 10, 15, 0, 1);
70         event.addChoice("サボりながら楽しくや
71             る", -5, -5, 0, 1);
72         event.addChoice("幽霊部員になる", 0, -5, 0, 1);
73         break;
74     case 14:
75         event = new LifeEvent("【14歳】中二病の季節。");

```

```

65         event.addChoice("深夜ラジオに投稿する", -5, -5, 0, 1);
66         event.addChoice("ポエムを書き溜める", 0, 0, 0, 1);
67         event.addChoice("悪い先輩とつるむ", -10, 5, 0, 1);
68         break;
69     case 15:
70         event = new LifeEvent("
71             【15歳】高校受験。人生最初の試練。");
72         event.addChoice("トップ高を目指して猛勉
73             強", -10, 40, -200000, 1);
74         event.addChoice("確実な学校を選ぶ", 5, 10, 0, 1);
75         event.addChoice("勉強より思い出作り", 0, -10, 0, 1);
76         break;
77     case 16:
78         event = new LifeEvent("
79             【16歳】高校入学。環境の変化。");
80         event.addChoice("バイトを始めて社会を知
81             る", -5, 10, 300000, 1);
82         event.addChoice("帰宅部でゲーム三昧", -5, -10, 0, 1);
83         event.addChoice("恋人を作る!", 5, 20, -50000, 1);
84         break;
85     case 17:
86         event = new LifeEvent("【17歳】修学旅行の夜。");
87         event.addChoice("好きな人の話で盛り上が
88             る", 5, -5, 0, 1);
89         event.addChoice("先生に見つかり正座", -5, 10, 0, 1);
90         event.addChoice("一人で早く寝る", 5, -5, 0, 1);
91         break;
92     case 18:
93         event = new LifeEvent("
94             【18歳】進路選択。大人への第一歩。");
95         event.addChoice("大学進
96             学(奨学金なし)", 0, 10, -5000000, 1);
97         event.addChoice("大学進
98             学(奨学金あり)", 0, 20, -1000000, 1);
99         event.addChoice("就職して自立", -5, 30, 2000000, 5);
100         break;
101     case 19:
102         event = new LifeEvent("
103             【19歳】大学1年。サークルの新歓。");
104         event.addChoice("飲みサーでウェイウェ
105             イ", -10, -10, -100000, 1);
106         event.addChoice("真面目な研究会に入る", 0, 5, 0, 1);
107         break;
108     case 20:
109         event = new LifeEvent("
110             【20歳】成人式。地元の友人と再会。");
111         event.addChoice("朝まで飲み明か
112             す", -10, -5, -30000, 1);
113         event.addChoice("行かずに家で過ごす", 5, -5, 0, 1);

```

```

102         break;
103     case 21:
104         event = new LifeEvent("
105             【21歳】就職活動。お祈りメール。");
106         event.addChoice("自己分析をやり直す", -5, 10, 0, 1);
107         event.addChoice("面接を受けまく
108             る", -10, 30, -50000, 1);
109         event.addChoice("現実逃避して旅に出
110             る", 5, -20, -200000, 1);
111         break;
112     case 22:
113         event = new LifeEvent("
114             【22歳】卒業論文と卒業旅行。");
115         event.addChoice("卒論を完璧に仕上げ
116             る", -10, 20, 0, 1);
117         event.addChoice("海外へ卒業旅
118             行", 5, -10, -300000, 1);
119         break;
120     default:
121         event = new LifeEvent("【" + age + "歳】時は流れま
122             す...");
123         event.addChoice("次へ進む", 0, 0, 0, 1);
124         break;
125 }
126 return event;
127 }
128
129 // 23歳～64歳：社会人時代（ランダムイベント）
130 private LifeEvent getAdultEvent(int age) {
131     LifeEvent event = new LifeEvent("社会人生活");
132     double rnd = Math.random();
133
134     if (age < 35) {
135         if (rnd < 0.4) {
136             event = new LifeEvent("【" + age + "歳】仕事で大きなミ
137                 ス...");
138             event.addChoice("正直に謝る", 0, 20, 0, 3);
139             event.addChoice("隠蔽しようとする", -5, 40, 0, 3);
140             event.addChoice("同僚のせいにする", -5, 10, 0, 3);
141         } else if (rnd < 0.7) {
142             event = new LifeEvent("【" + age + "歳】友人たちが結婚
143                 していく。");
144             event.addChoice("婚活パーティーに参
145                 加", -5, 10, -50000, 3);
146             event.addChoice("仕事こそが恋
147                 人", -10, 20, 500000, 3);
148             event.addChoice("一人の時間を楽し
149                 む", 5, -10, -100000, 3);
150         } else {

```

```

139         event = new LifeEvent("【" + age + "歳】初めての大きな
        昇進チャンス!");
140         event.addChoice("休日返上で働
        く", -20, 30, 2000000, 3);
141         event.addChoice("ワークライフバランス重
        視", 5, -5, 500000, 3);
142     }
143 } else if (age < 50) {
144     if (rnd < 0.4) {
145         event = new LifeEvent("【" + age + "歳】マイホーム購入
        を検討中。");
146         event.addChoice("35年ローンで一軒家", 5, 30,
        -40000000, 4);
147         event.addChoice("都心の中古マンショ
        ン", 5, 10, -25000000, 4);
148         event.addChoice("気楽な賃貸暮ら
        し", 0, -5, -2000000, 4);
149     } else if (rnd < 0.7) {
150         event = new LifeEvent("【" + age + "歳】子供の反抗期と
        教育費。");
151         event.addChoice("塾に課金す
        る", -5, 10, -3000000, 4);
152         event.addChoice("家族旅行で絆を深め
        る", 5, -5, -500000, 4);
153         event.addChoice("放任主義", 0, 10, 0, 4);
154     } else {
155         event = new LifeEvent("【" + age + "歳】中間管理職の悲
        哀。");
156         event.addChoice("心を無にして耐え
        る", -5, 20, 1000000, 4);
157         event.addChoice("部下を飲みに連れて行
        く", -10, 5, -200000, 4);
158         event.addChoice("副業を始め
        る", -10, 10, 2000000, 4);
159     }
160 } else {
161     if (rnd < 0.4) {
162         event = new LifeEvent("【" + age + "歳】親の介護が必要
        になってきた。");
163         event.addChoice("同居して介護す
        る", -20, 40, 1000000, 5);
164         event.addChoice("施設にお願いす
        る", 5, 10, -10000000, 5);
165         event.addChoice("兄弟に任せる", 0, 20, 0, 5);
166     } else if (rnd < 0.7) {
167         event = new LifeEvent("【" + age + "歳】健康診断
        でE判定が出た。");
168         event.addChoice("お酒・タバコをやめ
        る", 10, 20, 500000, 5);
169         event.addChoice("気にせず豪遊す
        る", -20, -10, -1000000, 5);
170         event.addChoice("高いサプリを買

```

```

171         う", 0, 0, -500000, 5);
172     } else {
173         event = new LifeEvent("【" + age + "歳】役職定年。給料
174             が下がる。");
175         event.addChoice("プライドを捨てて働
176             く", -5, 10, 500000, 5);
177         event.addChoice("早期リタイアして田舎
178             へ", 10, -10, -2000000, 5);
179         event.addChoice("会社にしがみつ
180             く", -10, 30, 1000000, 5);
181     }
182 }
183 return event;
184 }
185
186 // 65歳～99歳：老後
187 private LifeEvent getSeniorEvent(int age) {
188     LifeEvent event = new LifeEvent("老後の日々");
189     double rnd = Math.random();
190
191     if (age < 75) {
192         if (rnd < 0.3) {
193             event = new LifeEvent("【" + age + "歳】再雇用終了。毎
194                 日が日曜日です。");
195             event.addChoice("ボランティアに参
196                 加", 5, -5, -10000, 3);
197             event.addChoice("シルバー人材で働
198                 く", -5, 5, 500000, 3);
199             event.addChoice("家でテレビ三昧", -10, 5, 0, 3);
200         } else if (rnd < 0.6) {
201             event = new LifeEvent("【" + age + "歳】孫が遊びに来ま
202                 した。");
203             event.addChoice("おもちゃを買い与え
204                 る", 5, -10, -100000, 3);
205             event.addChoice("昔話を聞かせる", 5, 0, 0, 3);
206             event.addChoice("疲れるので居留守を使う", 0, 5, 0, 3);
207         } else {
208             event = new LifeEvent("【" + age + "歳】夫婦で久しぶり
209                 の旅行。");
210             event.addChoice("豪華クルーズ世界一
211                 周", 10, -10, -5000000, 3);
212             event.addChoice("近場の温泉旅
213                 館", 5, -5, -100000, 3);
214         }
215     } else if (age < 90) {
216         if (rnd < 0.3) {
217             event = new LifeEvent("【" + age + "歳】運転免許の返納
218                 を迫られています。");
219             event.addChoice("素直に返納する", 0, -5, 0, 3);
220             event.addChoice("まだ運転できると怒る", 0, 10, 0, 3);
221         }
222     }
223 }

```

```

207         event.addChoice("電動カートを買
           う", 0, 0, -200000, 3);
208     } else if (rnd < 0.6) {
209         event = new LifeEvent("【" + age + "歳】親しい友人の葬
           式がありました。");
210         event.addChoice("自分の終活を始め
           る", 0, -5, -500000, 3);
211         event.addChoice("寂しさを紛らすため飲
           む", -10, -5, -20000, 3);
212         event.addChoice("死後の世界について考え
           る", 0, 10, 0, 3);
213     } else {
214         event = new LifeEvent("【" + age + "歳】足腰が弱ってき
           ました。");
215         event.addChoice("リハビリに励む", 5, 5, -100000, 3);
216         event.addChoice("老人ホーム入居を検
           討", 0, -10, -10000000, 3);
217         event.addChoice("家から出なくなる", -15, 0, 0, 3);
218     }
219     } else {
220         event = new LifeEvent("【" + age + "歳】
           100歳は目前です。今の心境は？");
221         event.addChoice("人生に感謝する", 10, -20, 0, 2);
222         event.addChoice("まだやり残したことがある", 5, 10, 0, 2);
223         event.addChoice("家族に「ありがとう」と伝え
           る", 10, -10, 0, 2);
224     }
225     return event;
226 }
227
228 private LifeEvent getEndingEvent() {
229     LifeEvent event = new LifeEvent("【100歳】大往生");
230     event.addChoice("静かに目を閉じる...", 0, 0, 0, 0);
231     return event;
232 }
233 }

```

ソースコード 9 GameLogic.java

```

1  /**
2   * Model層の窓口クラス。Controllerからの要求を一手に引き受ける。
3   */
4  class GameModel {
5      private final Player player;
6      private final GameLogic logic;
7
8      public GameModel() {
9          this.player = new Player();
10         this.logic = new GameLogic();

```

```

11     }
12
13     public Player getPlayer() {
14         return player;
15     }
16
17     public LifeEvent nextEvent() {
18         return logic.getEventForAge(player.getAge());
19     }
20
21     // 選択肢を適用し、結果（生存/死亡/クリア）を返す
22     public DeathResult applyChoice(Choice choice) {
23         // 1. ステータス反映
24         player.changeHealth(choice.healthDelta);
25         player.changeStress(choice.stressDelta);
26         player.changeMoney(choice.moneyDelta);
27
28         // 2. 死亡判定
29         player.checkVitality();
30         if (!player.isAlive()) {
31             return new DeathResult(
32                 DeathResult.Type.DEAD,
33                 "過労死・ストレス死",
34                 "あなたの人生はここで幕を閉じました。
35                  \n無理をしすぎたようです...",
36                 player.getAge()
37             );
38
39             // 3. 年齢加算
40             player.incrementAge(choice.ageDelta);
41
42             // 4. クリア判定
43             if (player.getAge() >= Player.HUMAN_LIFESPAN) {
44                 return new DeathResult(
45                     DeathResult.Type.DEAD,
46                     "大往生",
47                     "100年の人生を全うしました。
48                      \n素晴らしい人生でしたね。",
49                     player.getAge()
50                 );
51             }
52
53             // 5. 生存継続
54             return new DeathResult(DeathResult.Type.ALIVE, "", "",

```

```

54     }
55     }

```

ソースコード 10 GameModel.java (統合ファサード)

3. View 層 (GUI)

```

1  import javax.swing.*;
2  import java.awt.*;
3  import java.awt.event.ActionListener;
4  import java.util.List;
5
6  /**
7   * 画面表示を担当するクラス (Swing)
8   */
9  class GameView extends JFrame {
10     private final JLabel ageLabel = new JLabel();
11     private final JLabel statsLabel = new JLabel();
12     private final JTextArea eventArea = new JTextArea();
13     private final JPanel buttonPanel = new JPanel();
14
15     public GameView(ActionListener listener) {
16         setTitle("人生ログラン (0-100 歳)");
17         setSize(700, 600);
18         setDefaultCloseOperation(EXIT_ON_CLOSE);
19         setLayout(new BorderLayout());
20
21         // 上部パネル
22         JPanel topPanel = new JPanel(new GridLayout(2, 1));
23         topPanel.setBorder(BorderFactory.createEmptyBorder(10,
24             10, 10, 10));
25
26         ageLabel.setFont(new Font("Meiryo", Font.BOLD, 28));
27         ageLabel.setHorizontalAlignment(SwingConstants.CENTER);
28
29         statsLabel.setFont(new Font("Meiryo", Font.PLAIN, 16));
30         statsLabel.setHorizontalAlignment(SwingConstants.CENTER);
31
32         topPanel.add(ageLabel);
33         topPanel.add(statsLabel);
34         add(topPanel, BorderLayout.NORTH);
35
36         // 中央パネル

```



```

36     eventArea.setEditable(false);
37     eventArea.setLineWrap(true);
38     eventArea.setWrapStyleWord(true);
39     eventArea.setFont(new Font("Meiryo", Font.PLAIN, 18));
40     eventArea.setMargin(new Insets(30, 30, 30, 30));
41     add(new JScrollPane(eventArea), BorderLayout.CENTER);
42
43     // 下部パネル
44     buttonPanel.setLayout(new GridLayout(0, 1, 10, 10));
45     buttonPanel.setBorder(BorderFactory.createEmptyBorder(20,
46         20, 20, 20));
47     add(buttonPanel, BorderLayout.SOUTH);
48
49     setLocationRelativeTo(null);
50     setVisible(true);
51 }
52
53 public void updateDisplay(Player player) {
54     ageLabel.setText("年齢: " + player.getAge() + "歳");
55     String moneyColor = player.getMoney() < 0 ? "red" : "
56         black";
57     statsLabel.setText(String.format(
58         "<html>HP: %d | ストレス: %d | 所持金: <font color='%s'>%d円</font></html>",
59         player.getHealth(), player.getStress(), moneyColor,
60         player.getMoney()));
61 }
62
63 public void showEvent(LifeEvent event, ActionListener
64     listener) {
65     eventArea.setText(event.getText());
66     buttonPanel.removeAll();
67     List<Choice> choices = event.getChoices();
68     for (int i = 0; i < choices.size(); i++) {
69         JButton b = new JButton(choices.get(i).getText());
70         b.setFont(new Font("Meiryo", Font.PLAIN, 16));
71         b.setPreferredSize(new Dimension(0, 50));
72         b.setActionCommand(String.valueOf(i));
73         b.addActionListener(listener);
74         buttonPanel.add(b);
75     }
76     buttonPanel.revalidate();
77     buttonPanel.repaint();
78 }

```

```

76     public void showGameOver(String title, String message, int
       age) {
77         JOptionPane.showMessageDialog(this, message + "\n\
           n最終年齢:␣" + age + "歳", title, JOptionPane.
           PLAIN_MESSAGE);
78         System.exit(0);
79     }
80 }

```

ソースコード 11 GameView.java

4. Controller 層 (制御)

```

1  import java.awt.event.ActionEvent;
2  import java.awt.event.ActionListener;
3
4  /**
5   * ゲームの進行制御を担当するクラス
6   */
7  class GameController implements ActionListener {
8      private final GameModel model;
9      private final GameView view;
10     private LifeEvent currentEvent;
11
12     public GameController() {
13         model = new GameModel();
14         view = new GameView(this);
15         nextTurn();
16     }
17
18     // 次のターンへ進む
19     private void nextTurn() {
20         view.updateDisplay(model.getPlayer());
21         currentEvent = model.nextEvent();
22         view.showEvent(currentEvent, this);
23     }
24
25     // ボタンクリック時の処理
26     @Override
27     public void actionPerformed(ActionEvent e) {
28         int index = Integer.parseInt(e.getActionCommand());
29         if (index < 0 || index >= currentEvent.getChoices().size
           ()) return;
30

```

```
31         Choice selected = currentEvent.getChoices().get(index);
32         DeathResult result = model.applyChoice(selected);
33
34         if (result.type == DeathResult.Type.ALIVE) {
35             nextTurn();
36         } else {
37             view.updateDisplay(model.getPlayer());
38             view.showGameOver(result.title, result.message,
39                               result.age);
40         }
41     }
```

ソースコード 12 GameController.java